

Strict Evaluation and the Chunking Conjecture

Why we rewrote the B4 judge, and how prompt length affects candidate selection · Investor-call digital twins · June 2026

1. The problem: a lenient judge rewards topic-adjacency

The set-level question-prediction task is scored by an LLM judge (the “B4” evaluator) that compares each predicted question against the held-out actual analyst questions, assigns a 0–4 match score, and marks an actual as **covered** when its best match scores ≥ 3 . The original rubric (`b4_eval.md`) defined the scale purely by adjective — “3 = substantially similar, 4 = very close substitute” — leaving “substantially similar” to the model’s discretion.

In practice this lenient judge rewarded **topic adjacency**: a prediction in the same business area (yield, fuel, costs) was frequently scored ≥ 3 even when it asked a structurally different question or answered only part of the actual’s ask. The result is inflated coverage — and, more sharply, inflated precision — built on predictions that are not really the analyst’s question.

2. The fix: a strict, test-based rubric

We rewrote the rubric into an operational, test-based prompt (`b4_eval_strict.md`). The evaluator models, output schema, and prediction pools are unchanged, so the strict-vs-lenient difference is attributable to the rubric alone. Two changes carry the weight:

- **A substitution test gates score 3.** A prediction earns 3 only if, had management answered the predicted question verbatim, the listener would still obtain the key fact, number, or position the actual was asking for. Topic overlap alone is explicitly a 2, not a 3.
- **An ordered decision flow.** Score 4 requires a shared trigger phrase/number, the same ask shape (quantitative vs qualitative), and the same granularity (bps / \$ / narrative) — “an IR analyst reading both would not prepare separately for them.”

3. Why we need strict: false positives the lenient judge accepted

These are predictions the **lenient** judge scored ≥ 3 (covered) that do not actually answer the actual question. The strict substitution test demotes each one. (Scores shown as per-run lists; e.g. [2,1,3] is three runs.)

Actual analyst question (verbatim)	Matched prediction (verbatim) — lenient score	Strict outcome
sharon_zackfia-0: “Are you making any itinerary changes given higher fuel ... is there anything you’ve pulled back on this year ... <i>deferring to 2027</i> ? Is this just harvesting some of those efficiencies ...?”	sharon_zackfia-pred-0: “ <i>Net Yield was up 2% in Q1</i> ... can you quantify what portion ... is ticket price versus onboard/pre-cruise spend.” gpt-5-mini run-3 = 4 (a yield-decomposition ask — no overlap with the cost/deferral ask).	mini [1,2,2,1,1] → 0/5; gpt-5 [2,1,3] → 1/3. Answering the yield question tells the listener nothing about itinerary changes or deferred initiatives.
kevin_kopelman-0: “... North American customers and higher airfares ... have you seen any <i>consumer behavior change</i> ... ability to swallow those airfare increases ...?”	xian_siew-pred-6: “For the \$17.10–\$17.50 <i>EPS guidance</i> , can you provide the specific macro assumptions ... and the sensitivity of EPS to a 10% increase in average air travel costs to <i>Mediterranean</i> itineraries.” Scored 3–4 on the shared word “airfare.”	gpt-5 [2,1,2] → 0/3; mini 2/5 (from 3/5). Quantitative EPS-sensitivity vs qualitative demand, and a different region — the keyword no longer carries the match.

andrew_didora-0 (two parts): (1) "how do you think of *rolling in new hedges* in this high fuel environment?" (2) "at what level of *capacity growth* would we start to see more inflationary ... NCC ex-fuel growth ...?"

andrew_didora-pred-2: "On fuel: ... ~59% hedged ... \$0.62 EPS headwind ... hedging philosophy ... sensitivity ..." Scored **4** by mini in several runs — answers part (1) only.

gpt-5 [2,2,2] → **0/3**; mini **1/5**. A verbatim answer to the fuel question leaves part (2), the capacity-growth threshold, unanswered.

Another failure appears at large pools: at K=20 the lenient judge's argmax over ~217 candidates produces score-4 covers with **zero identity matches** — an actual "covered" only by *another twin's* prediction (e.g. lizzie_dove-0 covered by brandt_montour-pred-16). Strict's trigger and ask-shape requirements suppress most of these.

4. Strict is not merely punitive — genuine substitutes survive

- **Holds at 4:** vince_ciepiel-0 — the actual asks to decompose the +2% Q1 Net Yield; the prediction asks the same decomposition. Same trigger, ask shape, granularity, and analyst.
- **Holds at 3:** steven_wieczynski-0 — the prediction supplies the percent-booked-by-quarter and ticket-vs-onboard split that the actual's confidence question needs; the substitution test passes.

Aggregate effect: across all settings with both scores, the expected ordering strict ≤ lenient holds for **82 / 85** (the 3 exceptions are single-run judge noise). Strict coverage runs roughly **0.05–0.13** lower per setting; the tightening is concentrated on a handful of borderline actuals, not spread uniformly. The larger effect is on **precision**, where the lenient rubric's topic-adjacency inflation shows up most.

5. Chunking: shorter prompts help the judge pick a better question

Motivation. With the strict rubric fixed, a second problem is how the candidate pool is *presented* to the judge. With 150–220 candidates in one prompt, the judge does not reliably scan the whole list, misses covering candidates, and under-counts (e.g. v5 K20 single-call gpt-5-mini = 0.500, despite a pool that demonstrably covers more). This motivates a simple idea — **shorter prompts should improve candidate selection**: if the judge sees the pool in small chunks (≤50 candidates) instead of one long list, it can attend to each candidate and is more likely to surface the best match for each actual.

What happens after chunking. Empirically, chunking selects a *different, better* question than the single call rather than just re-scoring the same one. Two before/after cases (single-call → chunked, gpt-5 judge):

- matthew_boss-0 — "drivers of durable multi-year growth": the single call locked onto a capacity/deployment prediction (**score 2**); chunking found the Perfecta 20%-EPS-CAGR durable-growth prediction (**score 3**) — the on-topic match.
- sharon_zackfia-0 — "itinerary changes and deferred spend": the single call picked a Q2 cost-cadence prediction (**score 1**); chunking found the "Net Cruise Costs ex-fuel, structural vs timing" prediction (**score 3**).

In both, the long single call never reached the better candidate; the short chunk did. In aggregate (gpt-5 judge), the two methods pick a **different candidate in 66%** of (actual, config) pairs, and **76%** of the cases where chunking newly covers an actual come from a better candidate the long call never surfaced; chunking raises mean coverage **0.570 → 0.722** and wins 18 of 21 configs.

5a. The 2×2 of coverage (strict rubric, mean over 21 configs)

	single-call (long prompt)	chunked (short prompts)
gpt-5-mini	0.615 — under-counts (blind on large pools)	0.822 — over-counts (leniency × max-over-chunks)

gpt-5	0.570 — reliable but blind at high K	0.722 — reliable and accurate
--------------	--------------------------------------	--------------------------------------

The recommended estimate is the bottom-right cell. Chunked-mini is the *least* reliable — max-over-chunks gives a lenient judge several chances per actual, compounding into spurious 1.000 (all 12 covered) outliers that manual inspection refutes. Chunking must be paired with a reliable judge.

6. Takeaways

- **The strict prompt is necessary but not sufficient.** It encodes the substitution test, but gpt-5-mini applies it loosely (awards 3–4 to topic overlap that gpt-5 scores 2). Reliable evaluation needs the strict prompt *and* gpt-5.
- **Report coverage as a multi-run mean [min, max], not a single run**, and prefer gpt-5 as the canonical strict judge — the steadier scorer on exactly the borderline items where the rubric change matters.
- **The trustworthy coverage estimate is chunked retrieval scored by gpt-5** (mean ≈ 0.72 across 21 configs), well above the single-call gpt-5-mini K-curve (≈ 0.61) that under-counts through large-pool blindness.

7. Future work

The two results above are enabling steps toward a different goal: cleaning up the candidate pool itself. Once we have (1) a **more stable evaluation** — strict + gpt-5 read as a multi-run mean rather than a single noisy run — and (2) **better selection from large pools** via chunked retrieval, we can trust the judge enough to act on its verdicts. The natural next step is to **explore a filter that removes near-duplicate / overly similar questions from a large pool**: rather than letting the pool grow at high K (which lowers precision without raising strict coverage), use the same substitution test to detect when two candidates are mutually substitutable and drop the redundant one. This would raise precision while preserving coverage, and shrink the pool the judge must read — compounding the chunking benefit.

The chunking result is stated as a conjecture pending multi-run replication, a chunk-size sweep, and a two-stage retrieve-then-rescore control. Sources: `strict_eval_rewrite_report.md`, `FINAL_chunking_conjecture.md`.